Type-Directed Discretization of Probabilistic Programs

KATHERINE WU, Cornell University, USA
JULES JACOBS, Jane Street, USA
KEVIN BATZ, Cornell University, USA
ALEXANDRA SILVA, Cornell University, USA

We study *exact discretization* as a semantics-preserving the intended specification of a transformation for recursive, higher-order probabilistic programs with continuous distributions. We target programs where continuous values are compared against finitely many constants, so exact inference reduces the transformation reduces inference to a discrete problem. Our central technical contribution is a non-local, type-directed analysis that infers where continuous values can be partitioned into finitely many observationally relevant regions, then rewrites sampling and comparison behavior over those regions. We call this transformation Slice. Because this construction is global and type-directed, correctness requires reasoning beyond the local syntax: we formalize the transformation and prove soundness for boolean queries using a coupling-style logical relations argument over operational semantics. As an application, transformed its operational semantics, and state the measurable weak compiler simulation and terminal-limit compatibility needed for soundness of boolean queries. These proof obligations remain open. Transformed programs can be executed by discrete engines such as Dice, Roulette, and Storm. Our empirical evaluation shows two complementary strengths of that Slice when paired with discrete backends: it enables *exact inference* for supports case studies combining continuous sampling, higher-order functions, and unbounded recursion , and, on benchmarks where direct comparison is possible, it is competitive with state-of-the-art exact inference systems for continuous programs.

CCS Concepts: • **TODO**;

Additional Key Words and Phrases: probabilistic programming, exact inference, continuous distributions, discretization, type systems

ACM Reference Format:
Katherine Wu, Jules Jacobs, Kevin Batz, and Alexandra Silva. 2026. Type-Directed Discretization of Probabilistic Programs. 1, 1 (August 2026), 114 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 Introduction

Probabilistic programming languages (PPLs) are widely used to model uncertainty [6, 10, 14, 15, 17, 21, 24, 25, 27, 34, 35, 37, 39–41, 45–47]. Programs with *continuous* distributions capture rich models but require reasoning about uncountable domains, whereas purely *discrete* programs often admit fast and precise inference but cannot express real-valued models.

Slice studies *exact discretization*: a semantics-preserving source-to-source transformation that convertsintended to preserve semantics while converting, if possible, continuous programs into discrete ones. We target higher-order recursive programs with conditioning and continuous sampling, without arithmetic. The key mechanism is *type-directed cut inference*: our type system tracks each constant threshold (or *cut*) used in comparisons and uses this information to partition continuous domains into finitely many intervals. For example, $\texttt{uniform}(0,1) < 0.5$ discretizes to $\texttt{discrete}(0.5, 0.5) < 1$, yielding a discrete program that existing exact inference tools can process.

Cuts may arise far from their source expressions: a sampled value may flow through higher-order functions, conditionals, recursive calls, data structures, and observations before it is eventually compared. To discretize soundly, Slice propagates cut information along these non-local flows, collecting all observationally relevant cut points for each continuous value and using them to partition its domain into finitely many intervals

Authors' Contact Information: Katherine Wu, kaw324@cornell.edu, Cornell University, Ithaca, New York, USA; Jules Jacobs, julesjacobs@gmail.com, Jane Street, New York, New York, USA; Kevin Batz, ksb239@cornell.edu, Cornell University, Ithaca, New York, USA; Alexandra Silva, alexandra.silva@cornell.edu, Cornell University, Ithaca, New York, USA.

2026. ACM XXXX-XXXX/2026/8-ART
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

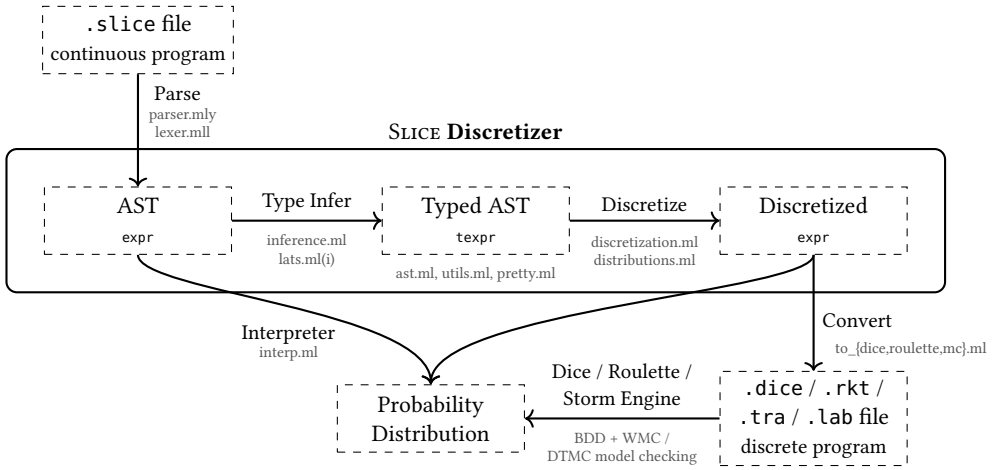


Fig. 1. Pipeline for exact discretization: typed Slice programs are transformed into semantically equivalent discrete programs, then evaluated by discrete engines. The transformation stage (type inference + discretization) is backend-agnostic; Dice, Roulette, and Storm are examples of target backends.

(see §2 for illustrative examples). A type system provides a natural framework for this analysis: it gives a compositional account of how cut information propagates through each language construct (via one typing rule per langauge feature); it separates the analysis result, i.e. a type derivation, from the type inference algorithm used to infer it; and it supports our correctness argument, since semantic preservation can be stated as a logical relation indexed by our types carrying cut and value sets.

Our discretization method is subject to a principled restriction: a continuous value must ultimately be compared against constants; direct comparisons between two unknown reals such as $\texttt{uniform}(0,1) < \texttt{uniform}(0,1)$ cannot be discretized exactly and remain continuous. Nonetheless, benchmarks from the literature and our novel case studies show that many programs *can* be discretized by our system automatically.

Discretization itself is not new. Garg et al. [20] encode real-valued variables via user-chosen bit-precisions, trading accuracy for tractability. Other approximate discretization schemes exist in the literature [5, 12, 28]. In Slice, *exact discretization* means that the transformation is semantics-preserving: the names the specification that the discrete program preserves the relevant probabilities of the original continuous program. This differs from approximate discretization, which approximates the original semantics and may provide instead targets error bounds or convergence guarantees, but does not prove equality of source and transformed program probabilities in the sense of our main theorem. The weak compiler simulation and terminal-limit compatibility needed to establish this specification remain open proof obligations.

Figure 1 shows the concrete pipeline from typed source programs to discrete backends. Because discretization is source-to-source, any discrete PPL that supports finite distributions can serve as a backend; we instantiate this with Dice [27], Roulette [35], and Storm [16].

Outline. Starting from examples (§2) and concluding with related and future work in §8, we:

Type System (§3) Characterize when programs are discretizable.

Type Inference (§4) Infer types automatically to guide discretization.

Exact Discretization (§5) Define a type-directed transformation from continuous programs to discrete ones using inferred type information.

Soundness (§6) Prove semantic preservation of the discretization transformation with a coupling-style relational argument Define the operational semantics and state the measurable weak compiler simulation and terminal-limit compatibility required to prove semantic preservation [7, 13, 48].

Implementation, Case Studies, and Evaluation (§7) Implement Slice type inference and the discretization transformation in OCaml, and provide Dice, Roulette, and Storm backends for running the discretized

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
respective (possibly overlapping) bounds *)
let init =
  (false, within 0.00 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 1.00) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Sample from a gaussian, then use
   the resulting value to update flags *)
let step = fun ys ->
  update (gaussian(0,1)) ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if uniform(0,1) < 0.1 then st
  else cover (step st) in

(* Have all intervals been covered at
   least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run entire process on sampled intervals *)
let final = cover init in
  is_covered final
```

Fig. 4. An infinite stochastic process for random interval covering over $[0, 1]$.

from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

Slice *automatically* produces a *provably equivalent* (cf. §6) discrete program (shown in §A.5). Even though this program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. Our key insight is that exact inference can then be performed via *probabilistic model checking*: we use Storm [16], which automatically computes that the sought-after probability is 0.034474.... See §7.1 for details. To the best of our knowledge, our pipeline is the first capable of performing exact inference on suitable programs containing continuous sampling, higher-order functions, and unbounded recursion. In §7.1.2, we extend this program by conditioning *inside the recursion*.

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct **observe**(e) conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one-

constraint is trivially satisfied. Otherwise, we again have a heuristic choice: we add a cut $<v$ for each value $v \in V_2$ (if $V_2 \neq \top$) and $\leq v$ for each value $v \in V_1$ (if $V_1 \neq \top$). If V_1 and V_2 are both $\top$, we set B to $\top$. (5)

- **Less-equal answerability** (answers$_{\leq}(B, V_1, V_2)$): Analogous to less-than. (5)
- **Equality constraints** ($B_1 = B_2$ or $V_1 = V_2$): We unify the variables, treating them as the same variable throughout the solving process. Any update to one variable is immediately reflected in all variables unified with it. (6)

The solver repeatedly processes constraints until reaching a fixed point, where no constraint causes any further updates to the variables. The monotonicity of the lattice operations ensures that this process terminates. This procedure may add more cuts to Bs than strictly necessary, but it never sets $B = \top$ unless that is the only way to satisfy all constraints. In other words, the discretization itself may be suboptimal in terms of the number of cuts.

Lattice constraint solving takes time polynomial in the size of the program: at worst, *every* constant in the program gets added to the cut set or value set of every subexpression of type $\mathbf{float}[B; V]$.

Additional constraint for contextual equivalence. When the return type of the program involves a float type, either directly or as part of a larger compound type (e.g., $\mathbf{float}[B; V] \to \mathbf{float}[B'; V']$), we add an additional constraint to ensure that these float types are not discretized. We recursively traverse the return type and set all nested B sets to $\top$. This ensures that the observable behavior of the program is preserved after discretization, regardless of the context in which the program is used. When the return type is basic, such as **bool**, nothing needs to be done.

Example 4.2. To make the constraint solving process concrete, Figure 12 walks through §2.3. Each constraint obtained from the generation phase is processed by the solver, then collected to obtain the final cut and value sets.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression e : **bool** and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}$ such that e and $\mathcal{D}[\![e : \mathbf{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this intended semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. Exactness here is the equality proved by Theorem 6.1 : the transformed program preserves the relevant probabilities of the source program, not merely an approximation with an error boundFor the uniform fragment defined in §6, Theorem 6.1 states this equality conditional on the remaining weak compiler-simulation obligation. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and exact discretization makes that reduction explicit.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; §6 proves that states the remaining proof obligation for this non-local, type-mediated constructionpreserves semantics. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e = \mathbf{uniform}(e_1, e_2)$ of type $\mathbf{float}[B; V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression $e' : \mathbf{fin}(n+1)$ such that the probability that e evaluates to a value in Intervals$(B)_i$ *coincides* with the probability that e' evaluates to $i_{\#n+1}$. In particular, if *all* $\mathbf{float}[B; V]$-typed subexpressions are finite-cut, then the given expression can be *fully discretized*.

Example 5.1. Consider a uniform distribution whose outcomes are compared to constants:

```
let x = uniform(0, 1) : float[B; T] in
  (x : float[B; T] < (0.3 : float[B; {0.3}])) ||
   (x : float[B; T] >= (0.5 : float[B; {0.5}]))
```

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \texttt{fin}(|B|+1) & \text{if } \tau = \texttt{float}[B;V] \text{ and } B \neq \top \\ \texttt{float}[B;V] & \text{if } \tau = \texttt{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

```
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating to 2.5 (resp. 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.*

6 Operational Semantics and Soundness

We now establish the soundness of our approach in the following sense: For *every* expression e : **bool**, possibly including *continuous sampling, higher-order functions, recursion, and conditioning,* The soundness statement in this section concerns the *uniform fragment* of Slice: the syntax in Figure 5 without sequencing, non-strict comparisons, finite comparisons in source programs, or continuous samplers other than **uniform**. We write $Expr^{\mathsf{U}}_{\tau,\mathrm{cl}}$ for its closed, well-typed, fully annotated expressions of type τ. This fragment still includes discrete choice, higher-order functions, unbounded recursion, lists, and conditioning. We write $Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$ for the smallest closed, well-typed operational carrier that additionally contains the finite comparisons and core-language dispatch expressions generated by Figure 13; for this source fragment, those comparisons are finite strict comparison and finite equality, and that is closed under the small-step rules below. This carrier definition assumes preservation of closedness and typing by capture-avoiding substitution, by the transformation, and by every small-step rule. Those metatheoretic properties are required premises, not consequences of the grammar alone. The small-step tables are exhaustive for $Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$; constructs in the full implementation outside this fragment are not covered by the theorem below. Write $Vals^{\mathsf{U}}_{\tau,\mathrm{cl}}$ and $Vals^{\mathsf{U}+}_{\tau,\mathrm{cl}}$ for the values in $Expr^{\mathsf{U}}_{\tau,\mathrm{cl}}$ and $Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$, respectively. The intended correctness statement is that, for every e : **bool** in $Expr^{\mathsf{U}}_{\textbf{bool},\mathrm{cl}}$,

e : **bool** *and* $\mathcal{D}[\![e : \textbf{bool}]\!]$: **bool** *(obtained from Figure 13) are semantically equivalent.*

The first step is to make formal sense of semantic equivalence. For that, we will define a monadic operational semantics for our language in §6.1. We will then proceed by proving the above claim. This is non-trivial: discretization Discretization decisions are driven by non-local, type-propagated information, so we cannot proceed by a local rule-by-rule argument . To prove the claim rigorously, we is insufficient. A complete proof must relate the executions of e and $\mathcal{D}[\![e]\!]$ through a coupling-style logical relation in §6.1. $\mathcal{D}[\![e]\!]$ through the weak compiler simulation stated below. That simulation remains an open proof obligation.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle : Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$, $\langle\cdot\rangle : Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} \to \mathcal{D}(Vals^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ in the operational closure to a *sub*-probability measure $\langle e : \tau \rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The probability mass in $\langle e : \tau \rangle$ is the probability that e diverges. The probability The probability of $\bot$ is the probability of encountering an observation failure. Missing mass represents either non-termination or a runtime parameter error. The implementation distinguishes these

cases operationally (by raising a runtime error for invalid sampling parameters), whereas this denotational model intentionally identifies them. With this model, semantic equivalence indeed captures our captures the sought-after correctness claim: If if $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!] \colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)[\![\cdot]\!] \colon Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} \to \mathcal{D}(Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \mathbf{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\mathbf{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; y : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel Subject to the carrier and skeleton-decomposition assumptions above, the measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* closed, well-typed operational expressions of type $\mathbf{bool}$ (i.e., over $Expr_{\mathbf{bool}}$) is then $Expr^{\mathsf{U}+}_{\mathbf{bool},\mathrm{cl}}$) is obtained from the measurable spaces of *all* the skeletons that give rise to expressions of type $\mathbf{bool}$ in $Expr^{\mathsf{U}+}_{\mathbf{bool},\mathrm{cl}}$.

Equipped with this family of measurable spaces, we define the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). For instance, if e_1 is not a value, then[1]

$$[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g. \begin{cases} \delta_\bot & \text{if } g = \bot \\ \delta_{(\mathbf{let}\ x = g\ \mathbf{in}\ e_2 : \tau_2) : \tau_2} & \text{otherwise.} \end{cases}}_{\text{standard probabilistic monadic bind over suitable sub-Markov kernels}},$$

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding $\mathbf{let}$-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by $\mathbf{observe}\ (\text{false} : \mathbf{bool}) : \mathbf{unit}$. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\mathbf{observe}\ (\text{false} : \mathbf{bool}) : \mathbf{unit}]\!](\{\bot\}) = 1$, the same holds for the surrounding $\mathbf{let}$-statement, i.e., also $[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. The Boolean operators use left-to-right short-circuit evaluation: false && e reduces to false and true || e reduces to true without evaluating e. All details — including proofs that all monadic binds are well-behaved since they operate on suitable Markov kernels — are provided in §F.4The appendix constructs the measurable expression spaces and records the remaining small-step measurability obligation needed to make these binds well-defined.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. For that, we first lift the one-step semantics $[\![\cdot]\!]$ Let the ordinary kernel S_τ take one step at non-values and return δ_v at a value v. The strict bind from Definition F.28 extends S_τ internally at $\bot$. We lift it to an n-step semantics $\langle\cdot\rangle_n$ in the expected wayby:

$$\langle e : \tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e : \tau\rangle_{n+1} \;=\; \langle e : \tau\rangle_n \gg\!= S_\tau$$

This construction is well-defined since the functional $[\![\cdot]\!]$ itself gives rise to a suitable Markov Conditional on the small-step functional being a sub-Markov kernel (Theorem F.39), this iteration is well-defined. We then define for every type τ the functional $\langle\cdot\rangle \colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$ as $\langle\cdot\rangle \colon Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} \to \mathcal{D}(Vals^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot)$ as follows. $\langle e : \tau\rangle = \lambda A.\, \lim_{n\to\infty} \langle e : \tau\rangle_n(A)$, which is well-defined by the monotone convergence theorem: the sequence $(\langle e : \tau\rangle_n)_{n\in\mathbb{N}}$ is monotonically increasing for every measurable set A over $Vals_\tau + \bot$ (cf. ??) and upper-bounded by 1: executing a program for more and more steps can

[1]Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A) \cdot \mu(dx)$, see Definition F.28 for details. δ_e denotes the Dirac distribution of e.

only increase the probability of terminating in a given set of values or an observation failureLet $T_\tau = Vals^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot$ and restrict each n-step measure to T_τ. Values and $\bot$ are absorbing, so these terminal restrictions form an increasing sequence of subprobability measures. We define $\langle e : \tau\rangle(A) = \sup_{n\in\mathbb{N}}\langle e : \tau\rangle_n(A)$ for measurable $A \subseteq T_\tau$. The supremum is a subprobability measure. This argument uses monotonicity only after restriction to terminal outcomes; the full n-step measures on expressions need not be monotone on arbitrary measurable sets.

6.2 Soundness via Probabilistic CouplingsThe remaining compiler-simulation obligation

We first state the central theorem, then outline our coupling-style correctness argument. Let $e : \mathbf{bool}$. Then $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$. Intuitively, we prove that the two programs can execute in a synchronized manner such that their probabilities of terminating in value v coincide for each $v \in \{\text{true}, \text{false}, \bot\}$.

Towards this end, define for every type τ the set $P_\tau = \{(e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e : \tau\}$. We define a *coupled small-step semantics* $[\![\cdot,\cdot]\!] : P_\tau \to \mathcal{D}(P_\tau + \bot)$ (cf. ????), which models the simultaneous execution of e and $\mathcal{D}[\![e : \tau]\!]$. The key property of this coupled semantics is that The ordinary rules above define the behavior of both source and generated expressions. They do not by themselves prove that discretization preserves behavior. In particular, the semantics of both e and $\mathcal{D}[\![e : \tau]\!]$ can be recovered in a sense we formalize nextgenerated term for a distribution with dynamic parameters takes deterministic administrative steps—let substitution, finite equality, Boolean conjunction, and conditional selection—for which the source has no matching step. A proof-only $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ operation that performs the complete dispatch in one step does not describe the term emitted by the transformation.

Given $\mu \in \mathcal{D}(P_\tau + \bot)$, we define the *projection of μ onto the i-th component* for $i \in \{1, 2\}$ as With projections, we recover the semantics of e and $\mathcal{D}[\![e : \tau]\!]$ as follows: Let $e : \tau$. We have:

$\pi_1([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![e : \tau]\!]$.

$\pi_2([\![e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![\mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]]\!]$.

$\Longrightarrow_{\mathrm{a}}$ be the reflexive transitive closure of those administrative target steps. The missing result is a measurable weak compiler-simulation relation R_τ between source expressions and generated target expressions. It must relate e to $\mathcal{D}[\![e]\!]$, allow either side to take finitely many administrative steps while the other stutters, couple every probabilistic step with the transition measures as exact marginals and support in R_τ, preserve terminal Boolean values, observation-failure mass, and missing mass, and rule out infinite administrative stuttering. The appendix states this obligation more precisely.

Now, using a limit construction analogous to §6.1.2 for obtaining a coupled *big-step* semantics $\langle\cdot,\cdot\rangle : P_\tau \to \mathcal{D}(V_\tau + \bot)$, where $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau \text{ is a value}\}$, we prove a suitable continuity property of projection (cf. ??) to conclude that ?? transfers to $\langle\cdot,\cdot\rangle$. Finally, observing that $V_{\mathbf{bool}} + \bot = \{(\text{true}, \text{true}), (\text{false}, \text{false}), \bot\}$ yields the desired claim:

Proof of Theorem 6.1. Let $v \in \{\text{true}, \text{false}, \bot\}$. We have

Conjecture 6.1 (Weak-simulation exactness). Suppose such a weak compiler simulation exists for every transformation clause in the uniform fragment, and suppose a terminal-limit compatibility theorem shows that its coupled finite-step approximants have equal terminal restrictions and that this equality is preserved by the suprema defining $\langle\cdot\rangle$. Then, for every $e : \mathbf{bool} \in Expr^{\mathsf{U}}_{\mathbf{bool},\mathrm{cl}}$,

$$\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle.$$

The earlier synchronous graph $\{(e, \mathcal{D}[\![e]\!])\}$ does not establish the hypothesis: it requires one source step and one target step at the same time, and it is not closed under the administrative reductions emitted for dynamic distribution parameters. Establishing the weak simulation remains an open proof obligation; therefore this paper does not presently claim unconditional exactness for the transformation.

7 Implementation, Case Studies, and Evaluation

We have implemented the Slice type inference and discretization procedures in OCaml. The implementation is open-source and available at https://github.com/[anonymized], together with all benchmarks, scripts, and a Docker image for reproduction. To compute the interval masses of continuous distributions, we use the OCaml GSL package [19] for cumulative distribution function calculations. All experiments were run in this Docker image on an Apple M3 with 16 GB of RAM.

This section is driven by two research questions:

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Discrete}\ \frac{}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)} \qquad \textsc{LessEq}\ \frac{\Gamma \vdash e : \mathbf{int}}{\Gamma \vdash e \le i : \mathbf{bool}} \qquad \textsc{Pair}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 * \tau_2} \qquad \textsc{Fst}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \mathbf{fst}\ e : \tau_1}$$

$$\textsc{Snd}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \mathbf{snd}\ e : \tau_2} \qquad \textsc{Fun}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathbf{fun}\ x\ \to\ e : \tau_1 \to \tau_2} \qquad \textsc{App}\ \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2} \qquad \textsc{True}\ \frac{}{\Gamma \vdash \mathrm{true} : \mathbf{bool}}$$

$$\textsc{False}\ \frac{}{\Gamma \vdash \mathrm{false} : \mathbf{bool}} \qquad \textsc{And}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \mathbf{bool}}{\Gamma \vdash e_1\ \&\&\ e_2 : \mathbf{bool}} \qquad \textsc{Or}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \mathbf{bool}}{\Gamma \vdash e_1\ ||\ e_2 : \mathbf{bool}}$$

$$\textsc{Not}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathrm{not}\ e : \mathbf{bool}} \qquad \textsc{Unit}\ \frac{}{\Gamma \vdash () : \mathbf{unit}} \qquad \textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}}$$

$$\textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{FinEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 ==_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}}$$

$$\textsc{Seq}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1; e_2 : \tau_2} \qquad \textsc{Fix}\ \frac{\Gamma, f : \tau_1 \to \tau_2, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \mathrm{fix}\ f\ x := e : \tau_1 \to \tau_2} \qquad \textsc{Nil}\ \frac{}{\Gamma \vdash \mathrm{nil} : \mathbf{list}(\tau)}$$

$$\textsc{Cons}\ \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \mathbf{list}(\tau)}{\Gamma \vdash e_1 :: e_2 : \mathbf{list}(\tau)} \qquad \textsc{Match}\ \frac{\Gamma \vdash e : \mathbf{list}(\tau_1) \qquad \Gamma \vdash e_1 : \tau_2 \qquad \Gamma, h : \tau_1, t : \mathbf{list}(\tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \mathrm{match}\ e\ \mathrm{with\ nil} \to e_1 \mid h :: t \to e_2\ \mathrm{end} : \tau_2}$$

$$\textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 25. Typing rules for general language constructs

F Soundness ProofObligations

Throughout this appendix, $Expr$ denotes the closed, well-typed operational carrier $Expr_{\mathrm{cl}}^{\mathrm{U}+}$ from §6; thus $Expr_\tau$ means $Expr_{\tau,\mathrm{cl}}^{\mathrm{U}+}$. Likewise, $Vals_\tau$ and $Skel_\tau$ denote, respectively, the values in this fragment and the skeletons arising from this fragment. An unqualified typing judgment $e : \tau$ in this appendix therefore ranges over $Expr_\tau$. Source-only domains are written explicitly as $Expr_{\mathrm{cl}}^{\mathrm{U}}$. We assume that capture-avoiding substitution, the transformation, and the small-step rules preserve closedness and typing, and that the fixed-skeleton decomposition applies to this carrier. The small-step kernel conjecture below records the remaining global measurability premise.

F.1 Measure Theory Preliminaries

We first give a brief overview of basic measure-theoretic terms and lemmas, and refer the reader to standard textbooks for further background [43, 44].

Lemma F.3 (Bijection induces isomorphism). *Let $f : X \to Y$ be a bijection and let (Y, Σ_Y) be a measurable space. Equip X with the σ-algebra generated by f:*

$$\Sigma_X \;:=\; f^{-1}(\Sigma_Y) \;=\; \{f^{-1}(A) \mid A \in \Sigma_Y\}.$$

Then,

(1) (X, Σ_X) is a measurable space, and
(2) f induces an isomorphism of measurable spaces:

$$(X, \Sigma_X) \cong (Y, \Sigma_Y).$$

Proof. We first verify that i) (X, Σ_X) is a measurable space, then show that ii) f induces an isomorphism of measurable spaces $(X, \Sigma_X) \cong (Y, \Sigma_Y)$.

Proof of i) We show that $\Sigma_X = f^{-1}(\Sigma_Y)$ is a σ-algebra on X.

- **(Contains X):** $X = f^{-1}(Y) \in \Sigma_X$ since $Y \in \Sigma_Y$.
- **(Complements):** If $f^{-1}(A) \in \Sigma_X$, then $X \setminus f^{-1}(A) = f^{-1}(Y \setminus A) \in \Sigma_X$ since $Y \setminus A \in \Sigma_Y$.
- **(Countable unions):** If $f^{-1}(A_i) \in \Sigma_X$ for $i \in \mathbb{N}$, then $\bigcup_i f^{-1}(A_i) = f^{-1}(\bigcup_i A_i) \in \Sigma_X$ since $\bigcup_i A_i \in \Sigma_Y$.

Hence (X, Σ_X) is a measurable space.

Proof of ii) Two measurable spaces (X, Σ_X) and (Y, Σ_Y) are isomorphic iff there exists a bijection $f : X \to Y$ such that f and f^{-1} are measurable.

We first show that f is measurable. Measurability of f means that $f^{-1}(A) \in \Sigma_X$ for every $A \in \Sigma_Y$. Let $A \in \Sigma_Y$ be arbitrary. By definition of Σ_X,

$$f^{-1}(A) \in f^{-1}(\Sigma_Y) = \Sigma_X.$$

It follows that f is measurable.

Next we show that f^{-1} is measurable. Measurability of f^{-1} means that $(f^{-1})^{-1}(B) \in \Sigma_Y$ for every $B \in \Sigma_X$. Let $B \in \Sigma_X$ be arbitrary. By definition of Σ_X, there exists some $A \in \Sigma_Y$ such that $B = f^{-1}(A)$.

Observe that:

$$\begin{aligned}
(f^{-1})^{-1}(B) &= f(B) && \text{(preimage of } f^{-1} \text{ is } f) \\
&= f(f^{-1}(A)) && \text{(substituting } B = f^{-1}(A)) \\
&= A && (f \text{ is a bijection})
\end{aligned}$$

Since $A \in \Sigma_Y$, it follows that f^{-1} is measurable.

Therefore f is a measurable bijection with measurable inverse, so it is an isomorphism of measurable spaces. □

Lemma F.4 (Minimality of generated σ-algebras). *Let X be a set and let $\mathcal{G} \subseteq \mathcal{P}(X)$. Suppose C is a σ-algebra on X such that $\mathcal{G} \subseteq C$. Then*

$$\sigma(\mathcal{G}) \subseteq C.$$

Proof. Recall that $\sigma(\mathcal{G})$ is defined as the smallest σ-algebra on X containing $\mathcal{G}$, given concretely by the intersection of all σ-algebras on X containing $\mathcal{G}$:

$$\sigma(\mathcal{G}) = \bigcap_{\substack{\mathcal{F} \text{ is a } \sigma\text{-algebra on } X \\ \mathcal{G} \subseteq \mathcal{F}}} \mathcal{F}.$$

Since C is a σ-algebra on X containing $\mathcal{G}$, it appears in this intersection. Therefore,

$$\sigma(\mathcal{G}) \subseteq C.$$

□

Definition F.5 (Sub-Markov kernels). Given measurable spaces (X, Σ_X) and (Y, Σ_Y), a Markov kernel *sub-Markov kernel* with source X and target Y is a function $P : X \times \Sigma_Y \to [0,1]$ (or equivalently, $X \to \Sigma_Y \to [0,1]$) that maps each source state $x \in X$ to a distribution over target states $P(x, -) : \Sigma_Y \to [0,1]$.

We call P a *Markov kernel* if it satisfies $P : X \times \Sigma_Y \to [0,1]$ (equivalently, $X \to \Sigma_Y \to [0,1]$) satisfying two conditions:

(1) *Probability* Subprobability *measure:* For each source state $x \in X$, the map $A \mapsto P(x, A)$ must be a probability $A \mapsto P(x, A)$ is a subprobability measure.
(2) *Measurable function:* For each event $A \in \Sigma_Y$ in the target space, the map $x \mapsto P(x, A)$ must be a measurable function.

Definition F.6 (Projection with error). Let $(X + \bot, \Sigma_{X+\bot})$ and $(Y + \bot, \Sigma_{Y+\bot})$ be measurable spaces, with (X, Σ_X) and (Y, Σ_Y) measurable spaces as well. Let $\mu \in \mathcal{D}((X \times Y) + \bot)$. We define the first and second projection measures as:

(1) $\pi_1(\mu)(A) := \mu(A \times Y)$ for every measurable $A \in \Sigma_X$, and $\pi_1(\mu)(\bot) := \mu(\bot)$
(2) $\pi_2(\mu)(B) := \mu(X \times B)$ for every measurable $B \in \Sigma_Y$, and $\pi_2(\mu)(\bot) := \mu(\bot)$

Then, $\pi_1(\mu)$ is a measure on $(X + \bot, \Sigma_{X+\bot})$ and $\pi_2(\mu)$ is a measure on $(Y + \bot, \Sigma_{Y+\bot})$.

F.2 Measure Space on Expressions

To make the probabilistic semantics precise, we construct an appropriate measure space structure on the set of expressions operationally closed fragment by factoring out real numbers using the notion of what we call *expression skeletons*.

Definition F.7 (Expression Skeleton). An expression skeleton s is a syntactic expression obtained from a program in the operationally closed fragment obtained by replacing each occurrence of a float constant with a hole of the form $\square$. The set of skeletons is generated by the grammar: following grammar (with only the constructs reachable from $Expr^{U+}$):

$$
\begin{aligned}
s ::= {} & \square \mid x \mid \text{true} \mid \text{false} \mid k_{\#n} \mid \textbf{diverge} \mid () \\
& \mid \textbf{let}\ x = s_1\ \textbf{in}\ s_2 \mid \textbf{if}\ s_1\ \textbf{then}\ s_2\ \textbf{else}\ s_3 \\
& \mid s_1 < s_2 \mid s_1 <_{\#n} s_2 \mid s_1 ==_{\#n} s_2 \\
& \mid \textbf{uniform}(s_1, s_2) \mid \textbf{discrete}(p_0, \ldots, p_{n-1}) \\
& \mid s_1\ \&\&\ s_2 \mid s_1\ ||\ s_2 \mid \text{not } s \mid \textbf{observe}(s) \\
& \mid (s_1, s_2) \mid \textbf{fst}\ s \mid \textbf{snd}\ s \\
& \mid \textbf{fun}\ x \to s \mid s_1\ s_2 \mid \textbf{fix}\ f\ x := s \\
& \mid \texttt{nil} \mid s_1 :: s_2 \\
& \mid (\textbf{match}\ s\ \textbf{with} \mid \texttt{nil} \to s_{\text{nil}} \mid h :: t \to s_{\text{cons}}\ \textbf{end})
\end{aligned}
$$

We denote by *Skel* the set of all expression skeletons, and by $holes(s)$ the number of distinct holes in skeleton s.

Example F.8. The expression $\textbf{uniform}(0.0, 1.0) < 0.5$ has the skeleton $\textbf{uniform}(\square, \square) < \square$ with three holes.

Definition F.9 (Filling a skeleton). Let $s \in Skel$ and let $\vec{r} = (r_1, \ldots, r_{holes(s)}) \in \mathbb{R}^{holes(s)}$. Define the expression $s[\vec{r}] \in Expr$ to be the result of replacing each hole $\square_i$ in s with the corresponding real constant r_i, for $1 \le i \le holes(s)$.

To recover a *typed* notion of skeletons, we extend the typing judgment to skeletons by adding a distinguished typing rule for holes.

Definition F.10 (Well-typed expression skeleton). Define a typing judgment for skeletons, written $\Gamma \vdash s : \tau$, with the understanding that all typing rules from expressions are preserved for skeletons, and that holes are typed as $\textbf{float}[B; V]$ according to the rule below:

$$\frac{}{\square : \mathbf{float}[B; V]}$$

We denote by $Skel_\tau$ the set of all well-typed expression skeletons in the operationally closed fragment, where:

$$Skel_\tau := \{s \in Skel \mid s : \tau\}.$$

Remark F.11. Both $Skel_\tau$ and $Expr_\tau$ are to be understood as the set of all well-typed skeletons and fragment skeletons and fragment expressions, respectively, *equipped with their full subtype information.*

For example,

$$\square : \mathbf{float}[B; \{0.5\}] < \square : \mathbf{float}[B; \{0.5\}] \quad \text{and} \quad \square : \mathbf{float}[B; \{0.2\}] < \square : \mathbf{float}[B; \{0.8\}]$$

are distinct elements of $Skel_{\mathbf{bool}}$, while

$$0.5 : \mathbf{float}[B; \{0.5\}] < 0.5 : \mathbf{float}[B; \{0.5\}] \quad \text{and} \quad 0.2 : \mathbf{float}[B; \{0.2\}] < 0.8 : \mathbf{float}[B; \{0.8\}]$$

are distinct elements of $Expr_{\mathbf{bool}}$.

In particular, both $Skel_\tau$ and $Expr_\tau$ are in general uncountable, since float subtype annotations of the form $\mathbf{float}[B; V]$ may vary over uncountably many real values.

Definition F.12 (Decomposing an expression). The decomposition function

$$\mathrm{dec} : Expr_\tau \longrightarrow \bigcup_{s \in Skel_\tau} \left(\{s\} \times \mathbb{R}^{holes(s)}\right)$$

is defined as the inverse of filling a skeleton, i.e., for every expression $e : \tau$,

$$\mathrm{dec}(e) = (s, \vec{r}) \quad \text{such that} \quad e = s[\vec{r}].$$

The explicit recursive definition of dec is shown in Figures 26 and 27.

Definition F.13 (Hole assignments of a skeleton). For a given $s : \tau \in Skel_\tau$, define the set of hole assignments of s as:

$$A_{s:\tau} := \bigcup_{e:\tau \in Expr_\tau} \{\mathrm{dec}(e : \tau)_2 \mid \mathrm{dec}(e : \tau)_1 = s : \tau\} \subseteq \mathbb{R}^{holes(s)}$$

Intuitively, $A_{s;\tau}$ is the set of all real tuples that can be substituted into the holes of s to yield a well-typed expression of type τ.

Example F.14. Consider the skeleton

$$s : \tau = (\square : \mathbf{float}[\{<3.5, <4.5\}; \top] \ < \ \square : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}.$$

This skeleton has two holes:

- The left hole has type $\mathbf{float}[\{<3.5, <4.5\}; \top]$, so it may be instantiated by any real number.
- The right hole has type $\mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]$, so it may only be instantiated by one of the values 3.5 or 4.5.

An expression $e : \mathbf{bool}$ has skeleton s iff it is of the form

$$e = c_1 < c_2$$

where $c_1 \in \mathbb{R}$ and $c_2 \in \{3.5, 4.5\}$.

Therefore, the admissible parameter tuples are precisely

$$A_{s:\tau} = \{(c_1, c_2) \mid c_1 \in \mathbb{R},\ c_2 \in \{3.5, 4.5\}\}.$$

Equivalently,

$$A_{s:\tau} = \mathbb{R} \times \{3.5, 4.5\} \subseteq \mathbb{R}^2.$$

Thus, expressions with this syntactic shape are parameterized by a two-dimensional space whose first coordinate ranges over $\mathbb{R}$ and whose second coordinate ranges over a finite set.

Proof. Let $s : \tau \in \mathit{Skel}_\tau$.

(i) Injectivity: Suppose $e_1, e_2 \in \mathit{Expr}_{s;\tau}$ satisfy $\mathrm{dec}_{s;\tau}(e_1) = \mathrm{dec}_{s;\tau}(e_2)$. By definition of $\mathrm{dec}_{s;\tau}$, $\mathrm{dec}_{s;\tau}(e_1) = \mathrm{dec}_{s;\tau}(e_2) = \vec{r}$ for some $\vec{r} \in A_{s;\tau}$. Since both e_1 and e_2 have skeleton s by assumption, we have $e_1 = s[\vec{r}]$ and $e_2 = s[\vec{r}]$, hence $e_1 = e_2$.

(ii) Surjectivity: Let $\vec{r} \in A_{s;\tau}$. By definition of $A_{s;\tau}$, there exists some expression $e \in \mathit{Expr}_\tau$ such that $\mathrm{dec}(e) = (s, \vec{r})$. Then $e \in \mathit{Expr}_{s;\tau}$ by construction of $\mathit{Expr}_{s;\tau}$, and $\mathrm{dec}_{s;\tau}(e) = \vec{r}$.

$\mathrm{dec}_{s;\tau}$ is injective and surjective, hence a bijection. □

Lemma F.21 (Isomorphism property). *For $s \in \mathit{Skel}_\tau$,*

$$(\mathit{Expr}_{s;\tau}, \mathcal{F}_{\mathit{Expr}_{s;\tau}}) \cong (A_{s;\tau}, \mathcal{F}_{s;\tau}).$$

Proof. We know $(A_{s;\tau}, \mathcal{F}_{s;\tau})$ is a Borel measurable space by Lemma F.16. First, verify that $\mathcal{F}_{\mathit{Expr}_{s;\tau}}$ coincides with the σ-algebra generated by $\mathrm{dec}_{s;\tau}$, i.e. $\mathcal{F}_{\mathit{Expr}_{s;\tau}} = \mathrm{dec}_{s;\tau}^{-1}(\mathcal{F}_{s;\tau})$:

$$\begin{aligned}
\mathcal{F}_{\mathit{Expr}_{s;\tau}} &:= \left\{\{e : \tau \in \mathit{Expr}_{s;\tau} \mid \mathrm{dec}(e : \tau)_2 \in B\} \mid B \in \mathcal{F}_{s;\tau}\right\} && \text{(Definition F.19)}\\
&:= \left\{\mathrm{dec}_{s;\tau}^{-1}(B) \mid B \in \mathcal{F}_{s;\tau}\right\} && \text{(unfolding the preimage of } \mathrm{dec}_{s;\tau})\\
&:= \mathrm{dec}_{s;\tau}^{-1}(\mathcal{F}_{s;\tau}) && \text{(Definition F.2)}
\end{aligned}$$

By Lemma F.20, $\mathrm{dec}_{s;\tau} : \mathit{Expr}_{s;\tau} \to A_{s;\tau}$ is a bijection. By Lemma F.3, $\mathrm{dec}_{s;\tau}$ induces an isomorphism $(\mathit{Expr}_{s;\tau}, \mathcal{F}_{\mathit{Expr}_{s;\tau}}) \cong (A_{s;\tau}, \mathcal{F}_{s;\tau})$. □

Lemma F.22. *$(\mathit{Expr}_{s;\tau}, \mathcal{F}_{\mathit{Expr}_{s;\tau}})$ is a Borel measurable space.*

Proof. By Lemma F.21. □

Now we combine the measurable spaces for each distinct skeleton into a single measurable space of all well-typed expressions in the operational closure of type τ. The set of all well-typed expressions of type τ is thus This set is the disjoint union:

$$\mathit{Expr}_\tau = \bigsqcup_{s:\tau \in \mathit{Skel}_\tau} \mathit{Expr}_{s;\tau}.$$

This union is disjoint because every typed expression $e : \tau$ has a unique skeleton, namely $\mathrm{dec}(e)_1$, so e belongs to exactly one set $\mathit{Expr}_{s;\tau}$.

Definition F.23 (σ-algebra construction on Expr_τ). Endow Expr_τ with the following σ-algebra:

$$\mathcal{F}_{\mathit{Expr}_\tau} := \{A \subseteq \mathit{Expr}_\tau \mid \forall s \in \mathit{Skel}_\tau.\ A \cap \mathit{Expr}_{s;\tau} \in \mathcal{F}_{\mathit{Expr}_{s;\tau}}\}$$

Lemma F.24. *$(\mathit{Expr}_\tau, \mathcal{F}_{\mathit{Expr}_\tau})$ is a measurable space.*

Proof. We verify that $\mathcal{F}_{\mathit{Expr}_\tau}$ is indeed a σ-algebra on Expr_τ.

(i) Contains Expr_τ: We show $\mathit{Expr}_\tau \in \mathcal{F}_{\mathit{Expr}_\tau}$. By Definition F.23, it suffices to show that $\forall s \in \mathit{Skel}_\tau$, $\mathit{Expr}_\tau \cap \mathit{Expr}_{s;\tau} \in \mathcal{F}_{\mathit{Expr}_{s;\tau}}$.

Observe that:

$$\begin{aligned}
&\mathit{Expr}_\tau \cap \mathit{Expr}_{s;\tau}\\
&= \mathit{Expr}_{s;\tau} && \text{(since } \mathit{Expr}_{s;\tau} \subseteq \mathit{Expr}_\tau)
\end{aligned}$$

Because $\mathcal{F}_{\mathit{Expr}_{s;\tau}}$ is a σ-algebra, it contains the set $\mathit{Expr}_{s;\tau}$. Hence,

$$\mathit{Expr}_{s;\tau} \in \mathcal{F}_{\mathit{Expr}_{s;\tau}}.$$

Therefore, $\mathit{Expr}_\tau \cap \mathit{Expr}_{s;\tau} \in \mathcal{F}_{\mathit{Expr}_{s;\tau}}$ for all $s \in \mathit{Skel}_\tau$. Therefore, $\mathit{Expr}_\tau \in \mathcal{F}_{\mathit{Expr}_\tau}$.

(ii) Closure under complements: Let $A \in \mathcal{F}_{Expr_\tau}$. We show that $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$. By Definition F.23, it suffices to show that $\forall s \in Skel_\tau$, $(Expr_\tau \setminus A) \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.

Fix $s \in Skel_\tau$. Observe that:

$$\begin{aligned}
&(Expr_\tau \setminus A) \cap Expr_{s;\tau} \\
&= (Expr_\tau \cap Expr_{s;\tau}) \setminus (A \cap Expr_{s;\tau}) && (\cap \text{ distributes over } \setminus) \\
&= Expr_{s;\tau} \setminus (A \cap Expr_{s;\tau}). && (\text{since } Expr_{s;\tau} \subseteq Expr_\tau, \text{ so } Expr_\tau \cap Expr_{s;\tau} = Expr_{s;\tau})
\end{aligned}$$

Since $A \in \mathcal{F}_{Expr_\tau}$ by assumption, by Definition F.23, we have $A \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$, and because $\mathcal{F}_{Expr_{s;\tau}}$ is a σ-algebra, it is closed under complements relative to $Expr_{s;\tau}$. Hence

$$Expr_{s;\tau} \setminus (A \cap Expr_{s;\tau}) \in \mathcal{F}_{Expr_{s;\tau}}.$$

Therefore $(Expr_\tau \setminus A) \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for all $s \in Skel_\tau$. Therefore, $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$.

(iii) Closure under countable unions: Let $(A_n)_{n\in\mathbb{N}}$ be a sequence with $A_n \in \mathcal{F}_{Expr_\tau}$ for all n. We show that $\bigcup_{n\in\mathbb{N}} A_n \in \mathcal{F}_{Expr_\tau}$. By Definition F.23, it suffices to show that $\forall s \in Skel_\tau$, $(\bigcup_{n\in\mathbb{N}} A_n) \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.

Fix $s \in Skel_\tau$. Using distributivity of $\cap$ over $\cup$, we have

$$\left(\bigcup_{n\in\mathbb{N}} A_n\right) \cap Expr_{s;\tau} = \bigcup_{n\in\mathbb{N}} \left(A_n \cap Expr_{s;\tau}\right).$$

Since each $A_n \in \mathcal{F}_{Expr_\tau}$ by assumption, by Definition F.23, we have $A_n \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for all n. Because $\mathcal{F}_{Expr_{s;\tau}}$ is a σ-algebra, it is closed under countable unions, hence

$$\bigcup_{n\in\mathbb{N}} \left(A_n \cap Expr_{s;\tau}\right) \in \mathcal{F}_{Expr_{s;\tau}}.$$

Therefore $(\bigcup_{n\in\mathbb{N}} A_n) \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for all s, and thus $\bigcup_{n\in\mathbb{N}} A_n \in \mathcal{F}_{Expr_\tau}$.

Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra on $Expr_\tau$, $(Expr_\tau, \mathcal{F}_{Expr_\tau})$ is a measurable space. □

Definition F.25 (Extension with error element). Let $(Expr_\tau, \mathcal{F}_{Expr_\tau})$ be the measurable space of well-typed operationally closed expressions of type τ. Define

$$Expr_\tau + \bot := Expr_\tau \cup \{\bot\},$$

equipped with the σ-algebra

$$\mathcal{F}_{Expr_\tau+\bot} := \{A,\ A \cup \{\bot\} \mid A \in \mathcal{F}_{Expr_\tau}\}.$$

Lemma F.26. *$(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ is a measurable space.*

Proof. We verify that $\mathcal{F}_{Expr_\tau+\bot}$ is indeed a σ-algebra on $Expr_\tau + \bot$.

By construction, every element of $\mathcal{F}_{Expr_\tau+\bot}$ is of the form A or $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$.

(i) Contains $Expr_\tau + \bot$: Take $A = Expr_\tau \in \mathcal{F}_{Expr_\tau}$. Then,

$$A \cup \{\bot\} = Expr_\tau \cup \{\bot\} = Expr_\tau + \bot \in \mathcal{F}_{Expr_\tau+\bot}.$$

(ii) Closure under complements: Let $C \in \mathcal{F}_{Expr_\tau+\bot}$. There are two cases:

- If $C = A$: Then, $(Expr_\tau + \bot) \setminus A = (Expr_\tau \setminus A) \cup \{\bot\}$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$, so $(Expr_\tau \setminus A) \cup \{\bot\}$ is of the form $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$ and hence lies in $\mathcal{F}_{Expr_\tau+\bot}$.
- If $C = A \cup \{\bot\}$: Then, $(Expr_\tau + \bot) \setminus (A \cup \{\bot\}) = Expr_\tau \setminus A$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $Expr_\tau \setminus A \in \mathcal{F}_{Expr_\tau}$, so $Expr_\tau \setminus A$ is of the form A for some $A \in \mathcal{F}_{Expr_\tau}$ and hence lies in $\mathcal{F}_{Expr_\tau+\bot}$.

In both cases the complement lies in $\mathcal{F}_{Expr_\tau+\bot}$.

(iii) Closure under countable unions: Let $(C_n)_{n\in\mathbb{N}}$ be a sequence with $C_n \in \mathcal{F}_{Expr_\tau+\bot}$ for all n. For each n, define $A_n := C_n \cap Expr_\tau \in \mathcal{F}_{Expr_\tau}$, where the latter holds since C_n is of the form A or $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$, and in either case $C_n \cap Expr_\tau = A$.

Let $A := \bigcup_{n\in\mathbb{N}} A_n$. Since $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $A \in \mathcal{F}_{Expr_\tau}$. Now observe:

$$\bigcup_{n\in\mathbb{N}} C_n = A \cup \left(\bigcup_{n\in\mathbb{N}} (C_n \setminus Expr_\tau)\right).$$

The second factor is either $\emptyset$ (if every $C_n = A_n$) or $\{\bot\}$ (if some $C_n = A_n \cup \{\bot\}$). In the first sub-case, $\bigcup_n C_n = A$ is of the form A for some $A \in \mathcal{F}_{Expr_\tau}$; in the second, $\bigcup_n C_n = A \cup \{\bot\}$ is of the form $A \cup \{\bot\}$ for some $A \in \mathcal{F}_{Expr_\tau}$. In both cases $\bigcup_n C_n \in \mathcal{F}_{Expr_\tau+\bot}$. □

Finally, we can equip this measurable space $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ with a subprobability measure such that $\mu(Expr_\tau + \bot) \leq 1$.

We give some basic lemmas:

Lemma F.27. *Let $(Expr_\tau + \bot, \mathcal{F}_{Expr_\tau+\bot})$ be the Borel measurable space of well-typed* operationally closed *expressions with error from Definition F.25. Let $Vals_\tau \subset Expr_\tau$ be the set of all well-typed value expressions* value expressions in this fragment. *Let $N_\tau \subset Expr_\tau$, defined as $N = Expr_\tau \setminus Vals_\tau$, be the set of all well-typed non-value expressions* in this fragment. *Let $\{\bot\}$ be the singleton set containing the error element $\bot$. The following are true:*

(1) The set $Vals_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.
(2) The set $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.
(3) The set $N_\tau \in \mathcal{F}_{Expr_\tau+\bot}$.

Proof. We prove the measurability of $Vals_\tau$, $\{\bot\}$, and N_τ in separate parts. For each, we show that it belongs to the σ-algebra $\mathcal{F}_{Expr_\tau+\bot}$ on the extended expression space $Expr_\tau + \bot$.

(1) **(Measurability of $Vals_\tau$):** Every expression skeleton $s \in Skel_\tau$ is either a value skeleton or a non-value skeleton, depending on whether the syntactic form of s is a value. An expression $e \in Expr_\tau$ is a value iff its skeleton is a value skeleton.
To show $Vals_\tau \in \mathcal{F}_{Expr_\tau}$, by Definition F.23, it suffices to show that $\forall s \in Skel_\tau, Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
Fix $s \in Skel_\tau$:
- If s is a value skeleton, then every expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$.
- If s is a non-value skeleton, then no expression in $Expr_{s;\tau}$ is a value, so $Vals_\tau \cap Expr_{s;\tau} = \emptyset \in \mathcal{F}_{Expr_{s;\tau}}$.

Thus, $Vals_\tau \cap Expr_{s;\tau} \in \mathcal{F}_{Expr_{s;\tau}}$ for any $s \in Skel_\tau$, so $Vals_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

(2) **(Measurability of $\{\bot\}$):** Taking $A = \emptyset$ in Definition F.25 gives $\{\bot\} \in \mathcal{F}_{Expr_\tau+\bot}$.

(3) **(Measurability of N_τ):** $N_\tau = Expr_\tau \setminus Vals_\tau$ is the complement of $Vals_\tau$ in $Expr_\tau$. Since $Vals_\tau \in \mathcal{F}_{Expr_\tau}$ and $\mathcal{F}_{Expr_\tau}$ is a σ-algebra, $N_\tau \in \mathcal{F}_{Expr_\tau} \subseteq \mathcal{F}_{Expr_\tau+\bot}$.

□

Note that in general, given (X, Σ_X) and (Y, Σ_Y) to be measurable spaces, $((X \times Y) + \bot)$ forms a measurable space, since the product $(X \times Y)$ is measurable and for any measurable space (Z, Σ_Z), the space $(Z + \bot, \Sigma_{Z+\bot})$ with $\Sigma_{Z+\bot}$ generated by $\Sigma_Z \cup \bot$ is measurable.

F.3 Distribution Monad with Error

Let X be a set equipped with a suitable measurable space so that $\mathcal{D}(X + \bot)$ denotes the set of probability subprobability measures over $X + \bot$. We will make heavy use of use the following generic monad construction, for which one easily verifies the monad lawsbind construction:

Definition F.28 (Distribution Monad with Error). We define

$$\text{return}\colon X \to \mathcal{D}(X + \bot), \quad \text{return}(x) = \delta_x$$
$$\text{bind}\colon \mathcal{D}(X + \bot) \to (X \to \mathcal{D}(Y + \bot)) \to \mathcal{D}(Y + \bot),$$
$$\text{bind}(\mu)(f)(A) = \int_{X+\bot} f_\bot(x)(A) \cdot \mu(dx)\,,$$

where δ_x denotes the Dirac measure for x, and f is a Markov kernel . an ordinary sub-Markov kernel on X, and $f_\perp$ is its strict extension from Definition F.29. Thus bind propagates existing $\perp$ mass.

Definition F.29 ($\perp$-extension). Define the $\perp$-extension of $g : X \to \mathcal{D}(Y + \perp)$ to be $g_\perp : (X + \perp) \to \mathcal{D}(Y + \perp)$, obtained from g by:

$$g_\perp(e) = \begin{cases} \delta_\perp & \text{if } e = \perp \\ g(e) & \text{otherwise .} \end{cases}$$

Lemma F.30 ($\perp$-extension is a sub-Markov kernel). *Given a Markov sub-Markov kernel $f : X \to \mathcal{D}(Y + \perp)$, its strict $\perp$ extension $f_\perp : (X + \perp) \to \mathcal{D}(Y + \perp)$ is also a Markov sub-Markov kernel.*

Proof. We verify the two conditions for $f_\perp$ to be a Markov sub-Markov kernel.

(1) Probability Subprobability measure: We show for each $e \in X + \perp$, the map

$$A \mapsto f_\perp(e)(A)$$

is a probability subprobability measure.

Fix $e \in X + \perp$.

- If $e = \perp$, then $f_\perp(\perp) = \delta_\perp$, the Dirac measure at $\perp$, which is a probability subprobability measure on $Y + \perp$.
- If $e \in X$, then $f_\perp(e) = f(e)$, and since f is a Markov kernel by assumption, $f(e)$ is a probability measure on $Y + \perp$which is a subprobability measure by assumption.

Thus, for all $e \in X + \perp$, the map $A \mapsto f_\perp(e)(A)$ is a probability subprobability measure.

(2) Measurable function: We show for each $A \in \mathcal{F}_{Y+\perp}$, the map $e \mapsto f_\perp(e)(A)$ is a measurable function.

Fix $A \in \mathcal{F}_{Y+\perp}$. Define

$$g\colon (X + \perp) \to [0, 1], \qquad g(e) := f_\perp(e)(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_{X+\perp}$ for every $B \in \mathcal{B}([0, 1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}((a, \infty) \cap [0, 1]) \in \mathcal{F}_{X+\perp}$$

because the family of sets $\{(a, \infty) \cap [0, 1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0, 1])$.

Because $g(e) \in [0, 1]$ for all $e \in X + \perp$ by definition, we have for every $a \in \mathbb{R}$,

$$\begin{aligned} g^{-1}\big((a, \infty) \cap [0, 1]\big) &= \{e \in X + \perp \mid g(e) \in (a, \infty) \cap [0, 1]\} \\ &= \{e \in X + \perp \mid (g(e) \in (a, \infty)) \wedge (g(e) \in [0, 1])\} \\ &= \{e \in X + \perp \mid (g(e) > a) \wedge (g(e) \in [0, 1])\} \\ &= \{e \in X + \perp \mid g(e) > a\}, \end{aligned}$$

where the last step uses that $g(e) \in [0, 1]$ always holds. Hence it suffices to show that $\{e \in X + \perp \mid g(e) > a\} \in \mathcal{F}_{X+\perp}$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{e \in X + \perp \mid g(e) > a\} = \emptyset \in \mathcal{F}_{X+\perp}$.
- If $a < 0$: $\{e \in X + \perp \mid g(e) > a\} = X + \perp \in \mathcal{F}_{X+\perp}$.
- If $0 \leq a < 1$: $\{e \in X + \perp \mid g(e) > a\}$.

The only nontrivial case is the last. It splits into two sets:

$$\begin{aligned} \{e \in X + \perp \mid g(e) > a\} &= \{e \in X + \perp \mid f_\perp(e)(A) > a\} \\ &= \{e \in X \mid f_\perp(e)(A) > a\} \cup \{\perp \mid f_\perp(e)(A) > a\} \\ &= \{e \in X \mid f(e)(A) > a\} \cup \{\perp \mid \delta_\perp(A) > a\} \end{aligned}$$

The first set is measurable in X (and hence $X + \perp$) since f is a Markov sub-Markov kernel by assumption. The second set is either $\emptyset \in \mathcal{F}_{X+\perp}$ or $\{\perp\} \in \mathcal{F}_{X+\perp}$. Since $\mathcal{F}_{X+\perp}$ is closed under countable unions, then $g^{-1}(B) \in \mathcal{F}_{X+\perp}$ for every $B \in \mathcal{B}([0, 1])$. Thus, for all $A \in \mathcal{F}_{Y+\perp}$, the map $e \mapsto f_\perp(e)(A)$ is a measurablefunctionHence the required inverse image is measurable.

Since both conditions hold, $f_\perp$ is a Markov sub-Markov kernel. □

Lemma F.31 (Dirac return is a sub-Markov kernel). *For any measurable space $(X, \mathcal{F}_X)$, the map*

$$\mathsf{return} : X \to \mathcal{D}(X + \bot), \qquad x \mapsto \delta_x.$$

is a Markov sub-Markov *kernel.*

Proof. We verify the two conditions for return to be a Markov kernel A Dirac probability measure is in particular a subprobability measure, and the measurability argument below establishes the second kernel condition.

(1) Probability measure: We show for each $x \in X$, the map

$$A \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a probability measure.

Fix $x \in X$. δ_x is the Dirac measure at x, hence a probability measure on $X + \bot$. Therefore, for every $x \in X$, the map $A \mapsto \delta_x(A)$ is a probability measure.

(2) Measurable function: We show that for each $A \in \mathcal{F}_{X+\bot}$, the map

$$x \mapsto \mathsf{return}(x)(A) = \delta_x(A)$$

is a measurable function.

Fix $A \in \mathcal{F}_{X+\bot}$. Define

$$g : X \to [0,1], \qquad g(x) := \delta_x(A).$$

We prove g is measurable by showing $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0,1])$. It suffices to prove that for every $a \in \mathbb{R}$,

$$g^{-1}\big((a,\infty) \cap [0,1]\big) \in \mathcal{F}_X,$$

because the family $\{(a,\infty) \cap [0,1] : a \in \mathbb{R}\}$ generates $\mathcal{B}([0,1])$.

Because $g(e) \in [0,1]$ for all $e \in X + \bot$, we have for every $a \in \mathbb{R}$,

$$\begin{aligned} g^{-1}\big((a,\infty) \cap [0,1]\big) &= \{e \in X \mid g(e) \in (a,\infty) \cap [0,1]\} \\ &= \{e \in X \mid (g(e) \in (a,\infty)) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid (g(e) > a) \wedge (g(e) \in [0,1])\} \\ &= \{e \in X \mid g(e) > a\}, \end{aligned}$$

where the last step uses that $g(e) \in [0,1]$ always holds. Hence it suffices to show that $\{e \in X \mid g(e) > a\} \in \mathcal{F}_X$ for every $a \in \mathbb{R}$.

- If $a \geq 1$: $\{x \in X \mid g(x) > a\} = \emptyset \in \mathcal{F}_X$.
- If $a < 0$: $\{x \in X \mid g(x) > a\} = X \in \mathcal{F}_X$.
- If $0 \leq a < 1$: $\{x \in X \mid g(x) > a\}$.

The only nontrivial case is the last. Using $g(x) = \delta_x(A)$ and the fact that $\delta_x(A) \in \{0,1\}$ for all $x \in X$, we have for $0 \leq a < 1$:

$$\begin{aligned} \{x \in X \mid g(x) > a\} &= \{x \in X \mid \delta_x(A) > a\} \\ &= \{x \in X \mid \delta_x(A) = 1\} \\ &= \{x \in X \mid x \in A\} \\ &= A \cap X. \end{aligned}$$

Since $A \in \mathcal{F}_{X+\bot}$ and $\mathcal{F}_X = \{X \cap A \mid A \in \mathcal{F}_{X+\bot}\}$ is the subspace σ-algebra on $X \subseteq X + \bot$, it follows that $A \cap X \in \mathcal{F}_X$.

Therefore $\{x \in X \mid g(x) > a\} \in \mathcal{F}_X$ for all $a \in \mathbb{R}$, so $g^{-1}\big((a,\infty) \cap [0,1]\big) \in \mathcal{F}_X$ for all a. Hence $g^{-1}(B) \in \mathcal{F}_X$ for every $B \in \mathcal{B}([0,1])$, and g is measurable. □

Lemma F.32 (Zero kernel is a sub-Markov kernel). *Let $(X, \mathcal{F}_X)$ and $(Y, \mathcal{F}_Y)$ be measurable spaces. The constant-zero map*

$$f : X \to \mathcal{D}(Y), \qquad f(x)(A) := 0$$

is a Markov sub-Markov *kernel.*

Proof. We verify the two kernel conditions.

(1) Probability Subprobability *measure.* We show for each $x \in X$, the map

$$A \mapsto f(x)(A)$$

is a probability subprobability measure.

Fix any $x \in X$. Then $f(x)$ is the zero measure: it satisfies σ-additivity since

$$f(x)\left(\bigcup_i A_i\right) = 0 = \sum_i 0 = \sum_i f(x)(A_i)$$

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

(2) Measurable function. For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on X, hence measurable.

Thus both conditions hold, so f is a Markov sub-Markov kernel. □

Lemma F.33 (Composition of sub-Markov kernels). *Let (X, Σ_X), (Y, Σ_Y), and (Z, Σ_Z) be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \text{and} \quad Q : Y \to \mathcal{D}(Z)$$

are Markov sub-Markov kernels, then their composition

$$(P \gg\!= Q)(x, A) := \int_Y Q(y, A)\, P(x, dy)$$

defines a Markov sub-Markov kernel

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. See [30, Lemma 1.38]. □

F.4 Small-Step Semantics

Throughout this section, we denote by $e : \tau$ $e : \tau$ denotes a well-typed expression of type τ and implicitly assume that all sub-expressions are annotated with types, as well. Moreover, for the typed expression $e : \tau = \texttt{uniform}(e_1 : \texttt{float}[B_1; V_1], e_2 : \texttt{float}[B_2; V_2]) : \texttt{float}[B, V]$ with $B, B_1, B_2 \neq \top$ and $V_1, V_2 \neq \top$, we introduce an auxiliary statement $\texttt{DC}_{B,B_1,B_2,V_1,V_2}$ and let We define the semantics of $\texttt{DC}_{B,B_1,B_2,V_1,V_2}$ in such a way that it behaves like the discretized program in the last case of Figure 13 but resolves the whose subexpressions are fully type-annotated. The dynamic-parameter transformation in Figure 13 denotes the displayed nest of core-language let-bindingsand the conditional choices in a single step, which will simplify our proof. We equip this statement with the following typing rule : , conditionals, Boolean conjunctions, finite equalities, and a discrete choice. No auxiliary one-step dispatch constructor is added to the language.

Equipped with these notions, we define the small-step semantics of our languagethe operationally closed fragment:

Definition F.34. The For $e : \tau \in Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}}$, the small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \bot)$ $[\![e : \tau]\!] \in \mathcal{D}(Expr^{\mathsf{U}+}_{\tau,\mathrm{cl}} + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2Tables 1 and 2.

The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., $\texttt{uniform}(0.8, 0.1)$). $[\![e : \tau]\!](\bot)$ is the probability of encountering an observation failure.

Lemma F.35 (Measurability of constant extension map). *Let $A \subseteq \mathbb{R}^n$ be equipped with the subspace σ-algebra*

$$\mathcal{F}_A := \{A \cap M \mid M \in \mathcal{B}(\mathbb{R}^n)\},$$

and let $B \subseteq \mathbb{R}^d$ be equipped with the subspace σ-algebra

$$\mathcal{F}_B := \{B \cap N \mid N \in \mathcal{B}(\mathbb{R}^d)\}.$$

Fix constant vectors $c_1 \in \mathbb{R}^{m_1}, \ldots, c_k \in \mathbb{R}^{m_k}$. Define

$$g : \mathbb{R}^n \to \mathbb{R}^d$$

Fix $A \in \mathcal{F}_{Expr_{\tau_2}}$. We know:

$$\begin{aligned}
\mathrm{Sub}_{e,x}^{-1}(A) &= \{v \in Val_{\tau_1} \mid e[v/x] \in A\} \\
&= \{v \in Val_{\tau_1} \mid e[v/x] \in A\} \cap Expr_{\tau_1} && (v \text{ ranges over } Val_{\tau_1} \subseteq Expr_{\tau_1}) \\
&= Val_{\tau_1} \cap \{u \in Expr_{\tau_1} \mid e[u/x] \in A\} && (\text{rewrite as } \cap \text{ with } Val_{\tau_1})
\end{aligned}$$

Now, define:

$$C := \{u \in Expr_{\tau_1} \mid e[u/x] \in A\}.$$

We claim proving $C \in \mathcal{F}_{Expr_{\tau_1}}$ is sufficient because by Definition F.1, this gives $\mathrm{Sub}_{e,x}^{-1}(A) = Val_{\tau_1} \cap C \in \mathcal{F}_{Val_{\tau_1}}$. To show $C \in \mathcal{F}_{Expr_{\tau_1}}$, by Definition F.23, it suffices to show that $\forall s \in Skel_{\tau_1}, C \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}}$.

Fix $s \in Skel_{\tau_1}$. We must prove:

$$\begin{aligned}
& C \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}} \\
\iff & \{u \in Expr_{\tau_1} \mid e[u/x] \in A\} \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}} && (\text{definition of } C) \\
\iff & \{u \in Expr_{s;\tau_1} \mid e[u/x] \in A\} \in \mathcal{F}_{Expr_{s;\tau_1}} && (Expr_{s;\tau_1} \subseteq Expr_{\tau_1}) \\
\iff & \{s[\vec{r}] \in Expr_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \in \mathcal{F}_{Expr_{s;\tau_1}} && (\text{rewrite } u) \\
\iff & \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \in \mathcal{F}_{s;\tau_1} && (\text{Lemma F.21})
\end{aligned}$$

Define

$$R_s := \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\}.$$

Let $t \in Skel_{\tau_2}$ be the skeleton obtained from e by replacing each occurrence of x with s. Define a constant extension map:

$$\psi_s : A_{s;\tau_1} \to A_{t;\tau_2}, \qquad \psi_s(\vec{r}) = \phi(\vec{r}, \vec{c_1}, ..., \vec{c_k})$$

where $\vec{c_1}, ..., \vec{c_k} \in \mathbb{R}$ are the float constants appearing in e (other than at occurrences of x), and ϕ permutes/interleaves the blocks into the order dictated by a left-to-right traversal of the holes of t.

More precisely, since t is obtained from e by replacing each free occurrence of x with s, the holes of t consist exactly of:

- the holes of s, repeated once per occurrence of x in e, and
- the holes coming from the float constants of e itself.

By Lemma F.35, ψ_s is measurable.

By construction, for all $\vec{r} \in A_{s;\tau_1}$:

$$e[s[\vec{r}]/x] = t[\psi_s(\vec{r})].$$

Define the measurable set of hole-assignments corresponding to A at skeleton t:

$$\begin{aligned}
A_t &:= \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A\} \\
&:= \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A \cap Expr_{t;\tau_2}\} && (t[\vec{u}] \text{ ranges over } Expr_{t;\tau_2})
\end{aligned}$$

By Definition F.23, $A \cap Expr_{t;\tau_2} \in \mathcal{F}_{Expr_{t;\tau_2}}$, and by Lemma F.3, $A_t \in \mathcal{F}_{t;\tau_2}$. We compute:

$$\begin{aligned}
R_s &= \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \\
&= \{\vec{r} \in A_{s;\tau_1} \mid t[\psi_s(\vec{r})] \in A\} \\
&= \{\vec{r} \in A_{s;\tau_1} \mid \psi_s(\vec{r}) \in A_t\} \\
&= \psi_s^{-1}(A_t).
\end{aligned}$$

Since ψ_s is measurable and $A_t \in \mathcal{F}_{t;\tau_2}$, we have $R_s = \psi_s^{-1}(A_t) \in \mathcal{F}_{s;\tau_1}$.

Since $R_s \in \mathcal{F}_{s;\tau_1}$ for every $s \in Skel_{\tau_1}$, we get $C \in \mathcal{F}_{Expr_{\tau_1}}$, and therefore

$$\mathrm{Sub}_{e,x}^{-1}(A) = C \cap Val_{\tau_1} \in \mathcal{F}_{Val_{\tau_1}}.$$

Hence $\mathrm{Sub}_{e,x}$ is measurable. □

Conjecture F.38 (Measurability of operational contexts). *For every occurrence of a monadic bind $\mu \gg\!= f$ given in the small-step operational semantics in Tables 1 and 2, $f : (Expr_{\tau_1} + \bot) \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov kernel.* sub-Markov kernel.

The skeleton construction reduces this claim to measurability of the corresponding context maps on each skeleton component. The argument is straightforward for fixed syntactic contexts, but a complete case analysis is not supplied here.

Conjecture F.39 (Small-step measurability). *For each type τ, the small-step operational semantics $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ defines a Markov kernel.* sub-Markov kernel.

This requires, in addition to Theorem F.38, measurability of the sampler kernels as their parameters vary and measurability of the case partition used by the small-step tables. These measure-theoretic obligations remain open.

F.5 Small-Step Coupling Big-Step Semantics

$e : \tau$ constraints $[\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]$

$\lambda A.0$ $v : \tau$ $\delta_{(v:\tau, \mathcal{D}[\![v:\tau]\!]:\mathcal{D}[\![\tau]\!])}$

$e_1 = v$ $\delta_{(e_2[v/x]:\tau_2, \mathcal{D}[\![e_2:\tau_2]\!][\mathcal{D}[\![v:\tau_1]\!]/x]:\mathcal{D}[\![\tau_2]\!])}$ e_1 no val.

$e_1 = \text{true}$ $\delta_{(e_2:\tau, \mathcal{D}[\![e_2:\tau]\!]:\mathcal{D}[\![\tau]\!])}$ $e_1 = \text{false}$ $\delta_{(e_3:\tau, \mathcal{D}[\![e_3:\tau]\!]:\mathcal{D}[\![\tau]\!])}$ o.w.

$e = \text{true}$ $\delta_{(\texttt{unit},\texttt{unit})}$ $e = \text{false}$ $\delta_\bot$ o.w.

$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $\delta_{(\text{true}:\texttt{bool},\text{true}:\texttt{bool})}$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $\delta_{(\text{false}:\texttt{bool},\text{false}:\texttt{bool})}$ $e_1 = v_1, e_2 \neq v_2$ o.w.

$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $\delta_{(\text{true}:\texttt{bool},\text{true}:\texttt{bool})}$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $\delta_{(\text{false}:\texttt{bool},\text{false}:\texttt{bool})}$ $e_1 = v_1, e_2 \neq v_2$ o.w.

$e_1 = v_1 \leq v_2 = e_2$ $\text{Uniform}(v_1, v_2) \gg\!= \lambda v.\delta_{(v:\texttt{float}[B;V], v:\texttt{float}[B;V])}$ $e_1 = v_1 > v_2 = e_2$ $[\![\texttt{diverge} : \tau, \mathcal{D}[\![\texttt{uniform}(v_1 : \tau_1, v_2 : \tau_2)]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $e_1 = v_1, e_2$ no val. e_1 no val.

$e_1 = c_1 \leq c_2 = e_2$ values $e_1 = v_1 > v_2 = e_2$ $[\![\texttt{diverge} : \tau, \mathcal{D}[\![\texttt{uniform}(v_1 : \tau_1, v_2 : \tau_2)]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $e_1 = c_1, e_2$ no val. e_1 no val. Small-step coupling semantics. Notice that $\mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]$ is uniquely determined by $e : \tau$ so that it suffices to indicate the latter in the left-hand column. Here, every function of the form $f(g, g') = \lambda(g, g').\delta_{\ldots}$ is extended to $f_\bot$ in the sense of Definition F.29.

$e : \tau$ constraints $[\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]$ $e_1 = b_1 = \text{true}, e_2 = b_2 = \text{true}$ $\delta_{(\text{true}:\texttt{bool},\text{true}:\texttt{bool})}$ $e_1 = b_1, e_2 = b_2$ $\delta_{(\text{false}:\texttt{bool},\text{false}:\texttt{bool})}$ $e_1 = b_1, e_2 \neq b_2$ o.w. $e_1 = b_1 = \text{false}, e_2 = b_2 = \text{false}$ $\delta_{(\text{false}:\texttt{bool},\text{false}:\texttt{bool})}$ $e_1 = b_1, e_2 = b_2$ $\delta_{(\text{true}:\texttt{bool},\text{true}:\texttt{bool})}$ $e_1 = b_1, e_2 \neq b_2$ o.w. $e = \text{true}$ $\delta_{(\text{false}:\texttt{bool},\text{false}:\texttt{bool})}$ $e = \text{false}$ $\delta_{(\text{true}:\texttt{bool},\text{true}:\texttt{bool})}$ o.w. $e_1 = v_1, e_2 = v_2$ $\delta_{((v_1,v_2):\tau_1 * \tau_2, (\mathcal{D}[\![v_1]\!]:\mathcal{D}[\![\tau_1]\!], \mathcal{D}[\![v_2]\!]:\mathcal{D}[\![\tau_2]\!]):\mathcal{D}[\![\tau_1]\!] * \mathcal{D}[\![\tau_2]\!])}$ $e_1 = v_1, e_2 \neq v_2$ o.w. $e = (v_1, v_2)$ $\delta_{(v_1:\tau_1, \mathcal{D}[\![v_1:\tau_1]\!]:\mathcal{D}[\![\tau_1]\!])}$ o.w. $e = (v_1, v_2)$ $\delta_{(v_2:\tau_2, \mathcal{D}[\![v_2:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!])}$ o.w. $\delta_{(\texttt{fun}\ x \to e:\tau_1 \to \tau_2, \texttt{fun}\ x \to \mathcal{D}[\![e]\!]:\mathcal{D}[\![\tau_1]\!] \to \mathcal{D}[\![\tau_2]\!])}$ $\delta_{(\texttt{fix}\ g\ x := e:\tau_1 \to \tau_2, \texttt{fix}\ g\ x := \mathcal{D}[\![e]\!]:\mathcal{D}[\![\tau_1]\!] \to \mathcal{D}[\![\tau_2]\!])}$ o.w. $\delta_{(\text{nil}:\texttt{list}(\tau), \text{nil}:\texttt{list}(\mathcal{D}[\![\tau]\!]))}$ $e_1 = v_1, e_2 = v_2$ $\delta_{(v_1::v_2:\texttt{list}(\tau), \mathcal{D}[\![v_1]\!]::\mathcal{D}[\![v_2]\!]:\texttt{list}(\mathcal{D}[\![\tau]\!]))}$ $e_1 = v_1, e_2 \neq v_2$ o.w. $e = \text{nil}$ $\delta_{(e_1:\tau_2, \mathcal{D}[\![e_1:\tau_2]\!]:\mathcal{D}[\![\tau_2]\!])}$ $e = v_h :: v_t$ $\delta_{(e_2[v_h/h, v_t/t]:\tau_2, \mathcal{D}[\![e_2:\tau_2]\!][\mathcal{D}[\![v_h]\!]/h, \mathcal{D}[\![v_t]\!]/t]:\mathcal{D}[\![\tau_2]\!])}$ o.w. Small-step coupling semantics (continued). Here, every function of the form $f(g, g') = \lambda(g, g').\delta_{\ldots}$ is extended to $f_\bot$ in the sense of Definition F.29.

Define $P_\tau = \{(e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]) \mid e : \tau\}$. Exploiting that $e : \tau$ is a value iff $\mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]$ is a value, we define: The Conditional on Theorem F.39, the complete ordinary small-step coupling $[\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!] \in \mathcal{D}(P_\tau + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in ????.

For every occurrence of a monadic bind $\mu \gg\!= (f, f')$ given in the small-step operational coupling semantics in ????, $(f, f') : (P_{\tau_1} + \bot) \to \mathcal{D}(P_{\tau_2} + \bot)$ is a Markov kernel.

Analogous to proof of Theorem F.38.

For each type τ, the small-step operational coupling semantics $[\![\cdot, \cdot]\!] : P_\tau \to \mathcal{D}(P_\tau + \bot)$ defines a Markov kernel .

Analogous to proof of Theorem F.39.

We now define suitable projection functions on measures $\mu \in \mathcal{D}(P_\tau + \bot)$ by tables above can be iterated without assuming a synchronous coupling. Define the ordinary kernel $S_\tau : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ to take one small step at non-values and to return δ_v at values. Its strict extension $(S_\tau)_\bot$ makes values and $\bot$ absorbing. With the strict bind from Definition F.28, define

$$\mu_0^e = \delta_e \qquad \text{and} \qquad \mu_{n+1}^e = \mu_n^e \gg\!= S_\tau.$$

We have the following key property: Let $e : \tau$. We have:

$\pi_1([\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![e : \tau]\!]$.

$\pi_2([\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]) = [\![\mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]$.

By induction on the type derivation of $e : \tau$. The most interesting case is where

$c_1 \leq c_2$,

$B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$,

$V = \top$,

$V_1 = \{v_1, \ldots, v_{n_1}\}$ with $\mathsf{recovers}(B_1, V_1)$ and $c_1 \in V_1$ due to $\mathsf{has}(V_1, c_1)$ by I. H.

$V_2 = \{w_1, \ldots, w_{n_2}\}$ with $\mathsf{recovers}(B_2, V_2)$ and $c_2 \in V_2$ due to $\mathsf{has}(V_2, c_2)$ by I. H.

First observe that we have where k_i is the unique number such that $c_i \in \mathsf{Intervals}(B_i)_{k_i}$. By Table 1 and Figure 13, this yields

For ??.??, consider the following:

For ??.??, consider the following:

Now, the remaining cases are either (i) trivial base cases or (ii) immediately follow from the I.H. and ?? further below. We demonstrate this for the case where

$e_1 = c_1$ is a value but e_2 is no value,

$B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$,

$V = \top$,

$V_1 = \{v_1, \ldots, v_{n_1}\}$ with $\mathsf{recovers}(B_1, V_1)$ and $c_1 \in V_1$ due to $\mathsf{has}(V_1, c_1)$ by I.H.

$V_2 = \{w_1, \ldots, w_{n_2}\}$ with $\mathsf{recovers}(B_2, V_2)$ and $c_2 \in V_2$ due to $\mathsf{has}(V_2, c_2)$ by I.H.

For ??.??, consider the following:

Let $F\colon P_\tau \to \mathcal{D}(P_\tau + \bot)$, let $G\colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$, and let $G'\colon Expr_{\mathcal{D}[\![\tau]\!]} \to \mathcal{D}(Expr_{\mathcal{D}[\![\tau]\!]} + \bot)$. We have:

If $\forall (f, f') \in P_\tau\colon F(f, f') = G(f)$, then for all $\mu \in \mathcal{D}(P_\tau + \bot)$,

If $\forall (f, f') \in P_\tau\colon F(f, f') = G'(f)$, then for all $\mu \in \mathcal{D}(P_\tau + \bot)$,

We only prove ??. ?? because the reasoning for ??.?? is completely analogous. We have

F.6 Big-Step Semantics and Correctness of Discretization

We extend both of the small-step semantics canonically to an n-step semantics by

As a sanity check, we show that $T_\tau = Vals_\tau + \bot$ and define the terminal restriction $\nu_n^e(A) = \mu_n^e(A \cap T_\tau)$. Absorption implies $\nu_n^e(A) \le \nu_{n+1}^e(A)$ for every measurable $A \subseteq T_\tau$. Notice that this statement is only about terminal restrictions: the n-step semantics defines a Markov kernel. full measures μ_n^e need not be monotone on arbitrary measurable sets.

For all $n \in \mathbb{N}$, $\langle\cdot\rangle_n : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel.

By induction on n.

Base case: $n = 0$. By definition,

i.e. the map $e \mapsto \delta_e$. By Lemma F.31, this is a Markov kernel.

Inductive step: $n > 0$. By definition,

By the I.H., $\langle\cdot\rangle_{n-1}\colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel. By Theorem F.39, $\lambda e'.[\![e' : \tau]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel. By Lemma F.30, its $\bot$-extension from $(Expr_\tau + \bot)$ to $\mathcal{D}(Expr_\tau + \bot)$ is also a Markov kernel. By Lemma F.33, $\langle e : \tau\rangle_{n-1} \gg\!= \lambda e'.[\![e' : \tau]\!]$ is a Markov kernel.

For all $n \in \mathbb{N}$, $\langle\cdot,\cdot\rangle_n : P_\tau \to \mathcal{D}(P_\tau + \bot)$ is a Markov kernel.

Analogous to proof of ??.

For every type τ, we denote by $Vals_\tau$ the set of values of type τ and equip it with a suitable measurable sub-space of the measurable space over $Expr_\tau + \bot$ (cf. Lemma F.27.1). Moreover, let $V_\tau = \{(v : \tau, \mathcal{D}[\![v]\!] : \mathcal{D}[\![\tau]\!]) \mid v : \tau\}$.

Let $e : \tau \in Expr_\tau$ and $n \in \mathbb{N}$. Let $Vals_\tau$ be the measurable set of values. We have:

$\langle e : \tau\rangle_n(Vals_\tau) \le \langle e : \tau\rangle_{n+1}(Vals_\tau)$

$\langle e : \tau\rangle_n(\bot) \le \langle e : \tau\rangle_{n+1}(\bot)$

By induction on n.

Base case: $n = 0$.

We must show $\langle e : \tau\rangle_0(Vals_\tau) \le \langle e : \tau\rangle_1(Vals_\tau)$.

By definition,

Therefore,

Now, for $n = 1$, by definition,

We analyze the two cases for e:

Case 1: e is a value v. Then, $\langle e : \tau\rangle_0(Vals_\tau) = 1$.

Definition F.40 (Big-step semantics). For $n = 1$: $e : \tau \in Expr_\tau$, define

$$\langle e : \tau\rangle(A) = \sup_{n \in \mathbb{N}} \nu_n^e(A) \qquad (A \subseteq T_\tau \text{ measurable}).$$

From the semantics, $[\![v : \tau]\!] = \delta_{v:\tau}$, which means $\langle e : \tau \rangle_1 = \delta_{v:\tau}$, and thus and thus $\langle e : \tau \rangle_0(Vals_\tau) \leq \langle e : \tau \rangle_1(Vals_\tau)$.

Case 2: e is an expression. Then, $\langle e : \tau \rangle_0(Vals_\tau) = 0$.

For $n = 1$: From the semantics, $[\![e : \tau]\!]$ may step to either a value, which means $[\![e : \tau]\!](Vals_\tau) \geq 0$, or an expression, which means $[\![e : \tau]\!](Vals_\tau) = 0$. In either case, and thus $\langle e : \tau \rangle_0(Vals_\tau) \leq \langle e : \tau \rangle_1(Vals_\tau)$.

Inductive step: We want to show that for $n = k + 1$:

By the recursive definition:

By the IH,

We do a case analysis on e' and e'', whether they are in N_τ or $Vals_\tau$.

Case $e' \in Vals_\tau, e'' \in Vals_\tau$.

Then, from the semantics, $[\![e']\!]$ and $[\![e'']\!]$ are both Diracs over a value. Combined with the IH and the fact that $[\![\cdot]\!]$ does not change the probability mass of $\cdot$ if it is a value,

Case $e' \in Vals_\tau, e'' \in N_\tau$.

Then, from the semantics, $[\![e']\!]$ is a Dirac over a value and $[\![e'']\!]$ is a distribution over a non-value expression. Combined with the IH and the fact that $[\![\cdot]\!]$ does not change the probability mass of $\cdot$ if it is a value and that $[\![\cdot]\!]$ does not add any probability mass to $Vals_\tau$ if $\cdot$ is Because $(\nu_n^e)_n$ is an increasing sequence of subprobability measures, its pointwise supremum is a non-value expression, subprobability measure. Missing mass represents execution that has not reached a value or observation failure in any finite number of steps, including divergence and runtime parameter errors in this model. This construction supplies the ordinary big-step semantics; it does not establish a relation between a source expression and its generated target.

Case $e' \in Vals_\tau, e'' \in N_\tau$.

F.6 Weak Compiler-Simulation Obligation

Analogous to the previous case, but with e' and e'' switched.

Case $e' \in N_\tau, e'' \in N_\tau$The transformation shown in Figure 13 emits core-language let-bindings, conditionals, Boolean conjunctions, finite equalities, and discrete choices. It does not emit the auxiliary $\mathsf{DC}_{B,B_1,B_2,V_1,V_2}$ constructor used by the earlier synchronous proof. Consequently, the graph $\{(e, \mathcal{D}[\![e]\!])\}$ is not closed under one ordinary step: target dispatch can take several deterministic steps while the source does not move.

Then, from the semantics, $[\![e']\!]$ and $[\![e'']\!]$ are both distributions over a non-value expression. Combined with the IH and the fact that $[\![\cdot]\!]$ does not add any probability mass to $Vals_\tau$ if $\cdot$ is a non-value expression,

In all cases, $\langle e : \tau \rangle_{k+1}(Vals_\tau) \leq \langle e : \tau \rangle_{k+2}(Vals_\tau)$.

Let $(e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]) \in P_\tau \longrightarrow_{\mathrm{a}}$ contain the deterministic administrative target reductions for let substitution, finite equality, Boolean conjunction, and conditional selection, and $n \in \mathbb{N}$. Let V_τ be the measurable set of value pairs. We have: let $\Longrightarrow_{\mathrm{a}}$ be its reflexive transitive closure. To obtain exactness, it remains to construct, for each type τ, a measurable relation R_τ such that:

(1) $\langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!] \rangle_n(V_\tau) \leq \langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!] \rangle_{n+1}(V_\tau)$ $(e, \mathcal{D}[\![e]\!]) \in R_\tau$ for every source expression in the uniform fragment;

(2) $\langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!] \rangle_n(\bot) \leq \langle e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!] \rangle_{n+1}(\bot)$

By induction on n. Analogous to proof of ??.

Let $e : \tau$. The *(big-step) semantics* $\langle e : \tau \rangle \in \mathcal{D}(Vals_\tau + \bot)$ and $\langle e : \tau, \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!] \rangle \in \mathcal{D}(V_\tau + \bot)$ are respectively defined as By ????, the sequence $\langle e : \tau \rangle_n$ is monotonically increasing for values and error. Moreover, $\langle e : \tau \rangle_n$ defines a subprobability measure, and thus $\langle e : \tau \rangle_n \leq 1$ for every $n \in \mathbb{N}$. Thus, by the monotone convergence theorem, the pointwise limit exists.

For all $n \in \mathbb{N}$, related expressions remain related after either target administrative reductions or matching source administrative reductions;

(3) $\pi_1(\lim_{n\to\infty} \langle e, e' \rangle_n) = \lim_{n\to\infty} \pi_1(\langle e, e' \rangle_n)$ every probabilistic step from either side has a coupling with the other side, after finitely many administrative steps, whose marginals are the corresponding subprobability transition measures and whose support is in R_τ;

(4) $\pi_2(\lim_{n\to\infty} \langle e, e' \rangle_n) = \lim_{n\to\infty} \pi_2(\langle e, e' \rangle_n)$

We prove part i) in detail; the proof of part ii) is analogous. Fix an arbitrary measurable $A \in \Sigma_1$: Since A was arbitrary, this implies: as desired.

For all $n \in \mathbb{N}$, related terminal Boolean values are equal, and both sides assign the same mass to observation failure and to missing mass;

```
let x1 = uniform(0,1) in
let x2 = if x1 < 0.56 then uniform(0,2) else uniform(0,3) in
let x3 = if x2 < 0.92 then uniform(0,4) else uniform(0,5) in
let x4 = if x3 < 0.47 then uniform(0,6) else uniform(0,7) in
let x5 = if x4 < 0.51 then uniform(0,8) else uniform(0,9) in
x5 < 0.5
```

Fig. 28. Conditional Independent (Figure 36a): Program of size 5. Each variable depends on its immediate predecessor variable via a conditional branch. Each uniform distribution is distinct.

(5) $\pi_1(\langle e:\tau, \mathcal{D}[\![e]\!]:\mathcal{D}[\![\tau]\!]\rangle_n) = \langle e:\tau\rangle_n$
$\pi_2(\langle e:\tau, \mathcal{D}[\![e]\!]:\mathcal{D}[\![\tau]\!]\rangle_n) = \langle \mathcal{D}[\![e]\!]:\mathcal{D}[\![\tau]\!]\rangle_n$ administrative reduction is normalizing on generated dispatch terms, so infinite administrative stuttering cannot hide terminal mass.

(6) the induced finite-step couplings have equal terminal restrictions, and this equality is preserved by the suprema defining the two big-step kernels.

By induction on n. We prove i), as the proof for ii) is analogous.

Base case: $n = 0$

Inductive step: $n > 0$

The following will imply our desired equivalence claim:

Let $\langle e:\tau, \mathcal{D}[\![e']\!]:\mathcal{D}[\![\tau']\!]\rangle \in \mathcal{D}(P_\tau + \bot)$. Then,

$\pi_1(\langle e:\tau, \mathcal{D}[\![e']\!]:\mathcal{D}[\![\tau']\!]\rangle) = \langle e:\tau\rangle$

$\pi_2(\langle e:\tau, \mathcal{D}[\![e']\!]:\mathcal{D}[\![\tau']\!]\rangle) = \langle \mathcal{D}[\![e']\!]:\mathcal{D}[\![\tau']\!]\rangle$

Conjecture F.41 (Weak-simulation exactness). *Let* e : **bool**. *Then* $\langle e$: **bool**$\rangle = \langle \mathcal{D}[\![e$: **bool**$]\!]$: **bool**$\rangle$. *Since* $\mathcal{D}(Vals_\tau + \bot)$ *contains only discrete distributions, we proceed by case distinction on the probability of the final outcome* $v \in \{\text{true}, \text{false}, \bot\}$.

The case $v = \text{true}$. *We have* A formal weak-simulation theorem satisfying the obligations above would imply that, for every e : **bool** $\in Expr^{\cup}_{\textbf{bool},\text{cl}}$,

$$\langle e : \textbf{bool}\rangle = \langle \mathcal{D}[\![e : \textbf{bool}]\!] : \textbf{bool}\rangle.$$

The cases $v = \text{false}$ *and* $v = \bot$ *are completely analogous.*

The synchronous coupling tables and their projection argument are therefore not part of the current proof. They relied on a proof-only $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ step and did not account for stuttering in the literal generated target. The conjecture above records the required theorem; its formal statement and proof remain open.

G Benchmarks

G.1 Asymptotic Scaling

G.1.1 Code Sketches for Scaling Benchmarks. This section contains code snippets of the programs used in the scaling experiments in Section 7.2. We implemented a Python script that automatically generates these programs based on configurable parameters such as program size, guard structure, and dependency pattern. For reproducibility, we used a fixed random seed to generate each program. All r_i values in the generated programs, where $0 \le r_i \le 1$ is randomly chosen, are rounded to two decimal places.

All of these programs follow the same structural pattern: they define a sequence of random variables $X_1, \ldots, X_n$, where each X_i is drawn from a uniform distribution whose parameters depend on the value of a single predecessor variable through a conditional branch of the form

$$X_i = \begin{cases} \text{Unif}(a_i, b_i), & X_{\pi(i)} < r_i, \\ \text{Unif}(a_i', b_i'), & \text{otherwise}, \end{cases}$$

for some parent-selection function $\pi : \{2, \ldots, n\} \to \{1, \ldots, i-1\}$. Thus each program is a Bayesian network of conditionally independent nodes, differing only in the choice of $\pi(i)$ that determines the dependency pattern.

The programs in Figure 28 follow the scheme depicted above where the program size is n, i.e., the number of comparisons performed is n. In Figure 29, each x_i does not necessarily depend on x_{i-1} but on some randomly